Tiling the Series

Outline

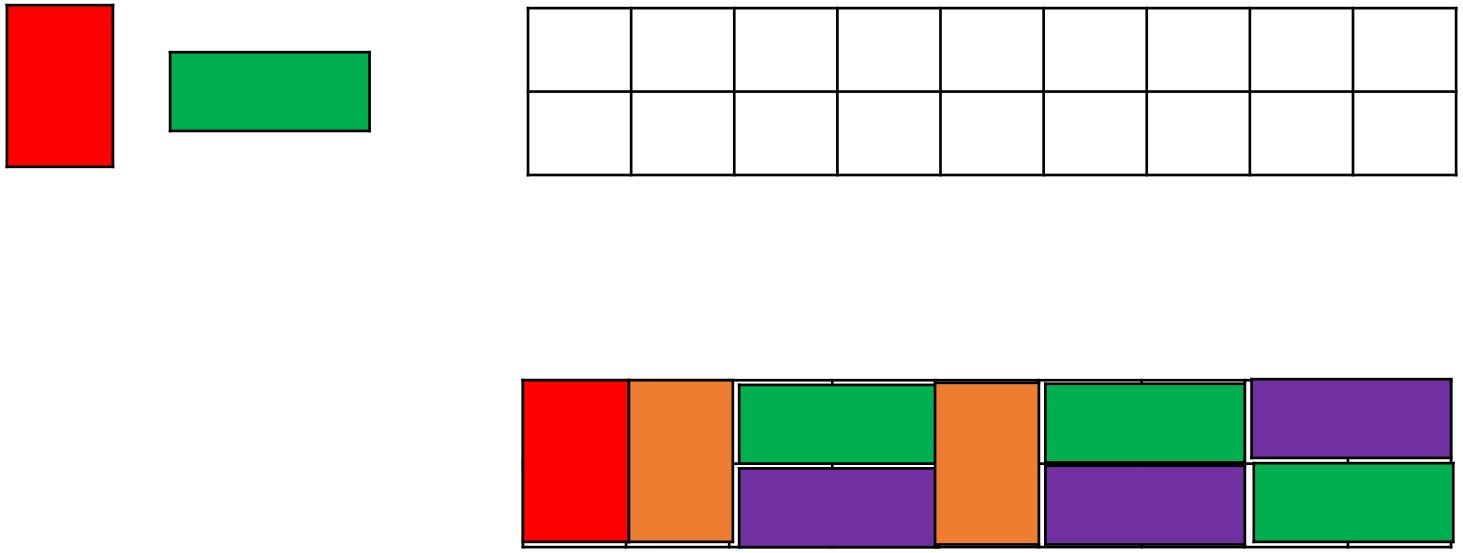
- Tiling v0 (ปู 1x2 ลงใน 2xm) ez
- Tiling v1 (ปู 1xm ลงใน nxm) ez
- Tiling v2 (ปู 1x2 ลงใน 3xn)
- Tiling v3 (tromino domino ลงใน 2xn)
- Tiling v4 (ปู เป็นสี่เหลี่ยมยาวเท่าไรก็ได้ ลงใน 2xn)
- Tiling v5 (ปู 1x2 ลงใน NxM)

Tiling V0

ให้ไปทำข้อ Tiling V0 ก่อน

กระเบื้อง V0

- **ข้อนี้มีพื้นที่ขนาด 2 x N ช่อง แล้วมีกระเบื้องขนาด 1x2 ช่อง**
- จงหาจำนวนวิธีในการปูตารางโดยใช้กระเบื้อง โดยที่กระเบื้องสามารถถูกวางได้ในแนวตั้งหรือแนวนอนเท่านั้น

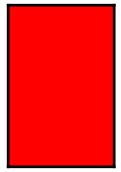


- **จากที่สังเกต เป็นโจทย์เรื่องการนับ**
  - เข้าข่ายเป็น Dynamic programming

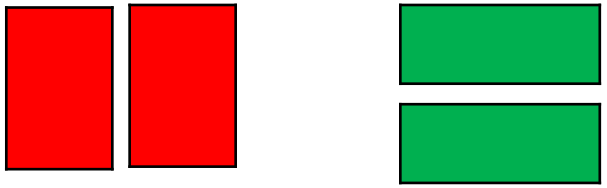
- **แล้วเราจะเริ่มอย่างไรดี**
  - เริ่มต้นด้วยกรณีง่ายๆ ก่อน

- ถ้าตารางมีขนาด 2 x 1 วางได้กี่แบบ
- ถ้าตารางมีขนาด 2 x 2 วางได้กี่แบบ
- ถ้าตารางมีขนาด 2 x 3 วางได้กี่แบบ
- ลองคิดดูสักครู่

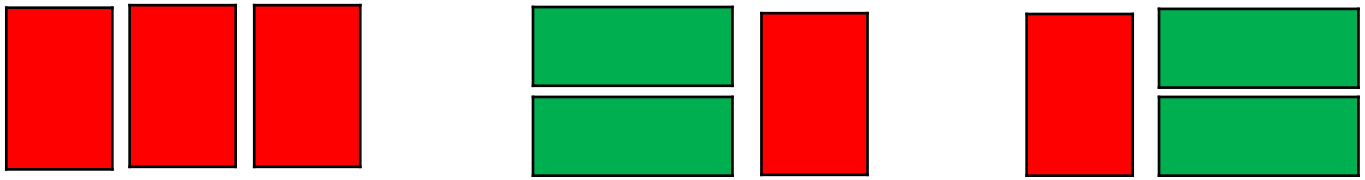
- ถ้า N = 1



- ถ้า N = 2



- ถ้า N = 3

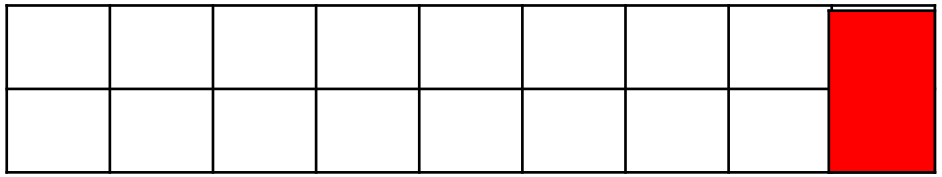


- ข้อสังเกตที่เราพบ

- ไม่ว่าจะวางแบบใด กระเบื้องแผนสุดท้ายจะปูไม่แนวตั้งก็แนวนอน

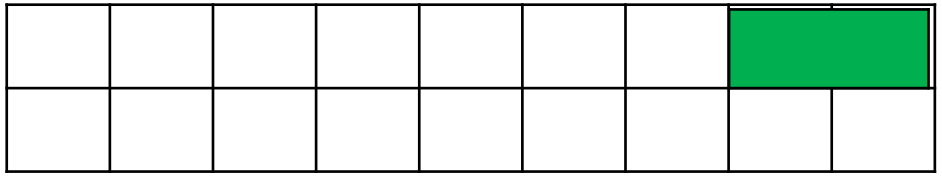
พิจารณาการวางตัวสุดท้าย วางได้อย่างไรบ้าง

- **ถ้าตัวสุดท้ายวางแนวตั้ง**



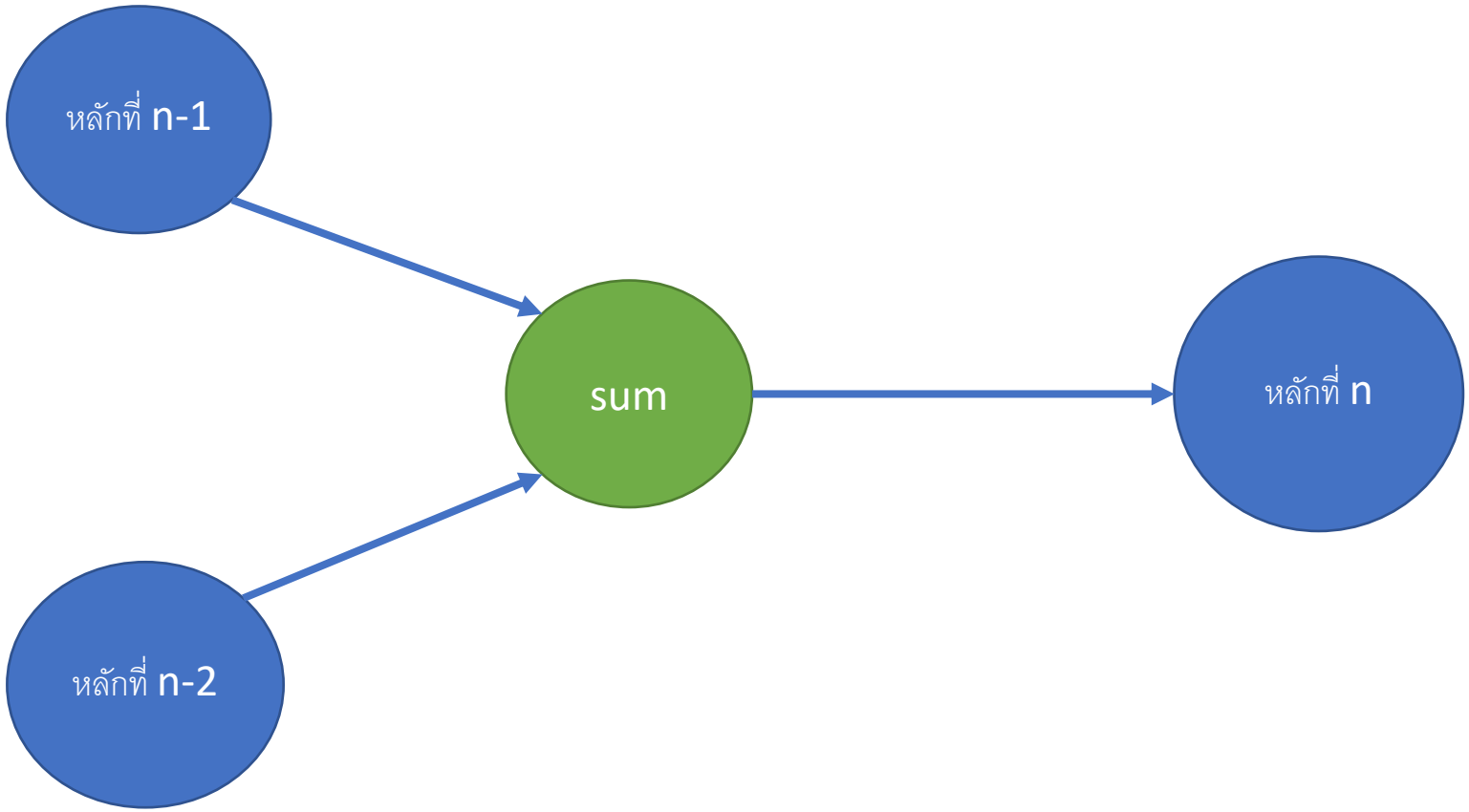
  - ไปถาม จำนวนการปูจากใครต่อ

- **ถ้าตัวสุดท้ายวางแนวนอน**



  - ไปถาม จำนวนการปูจากใครต่อ

- ดังนั้นพิจารณาหลักสุดท้าย วางได้อย่างไรบ้าง



- **เราได้กรณีทั่วไปแล้วนั่นคือ**
  - $T(n) = T(n-1) + T(n-2)$

- **Base case หละ**
  - ย้อนไปเรื่อยๆ มันจะหยุดตรงไหน หรือคิดว่า เริ่มต้นควรมีค่าเป็นแบบใด
  - นั่นคือการปู ตาราง 1 หลัก 2 หลัก 3 หลักที่ถามช่วงแรก

```
long T(n){
    if(n <= 2)
        return n;
    else
        return T(n-1) + T(n-2)
}
```

DP: Top Down

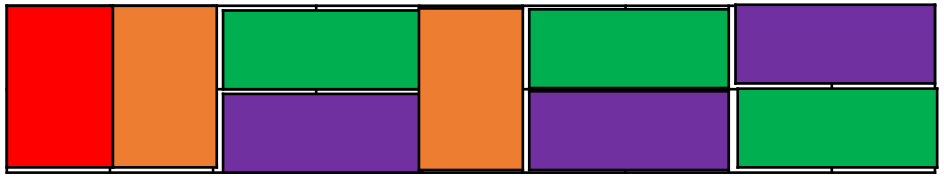
```
Initialize S[1..N] to -1
T(i) {
    if (S[i] != -1){
        return(S[i]);
    }else{
        if (i <= 2){
            S[i] = i;
            return(i);
        }
        if (i > 2){
            S[i] = T(i-1) + T(i-2);
            return(S[i]);
        }
    }
}
```

DP: Bottom up

```
T(i) {
        S[1] = 1;
        S[2] = 2;
        for (j = 3; j <= i; j++){
            S[j] = S[j-1] + S[j-2];
        }
        return(S[i]);

}
```

- วันนี้ก็ประมาณนี้ก่อน



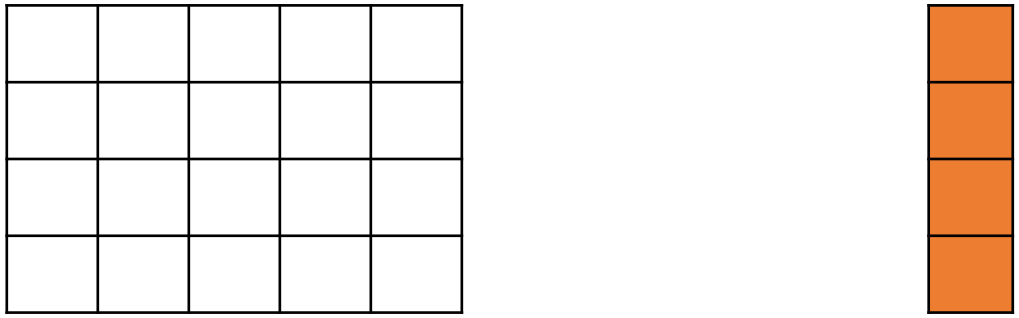
Outline

- Tiling v0 (ปู 1x2 ลงใน 2xm) ez
- Tiling v1 (ปู 1xm ลงใน nxm) ez
- Tiling v2 (ปู 1x2 ลงใน 3xn)
- Tiling v3 (tromino domino ลงใน 2xn)
- Tiling v4 (ปู เป็นสี่เหลี่ยมยาวเท่าไรก็ได้ ลงใน 2xn)
- Tiling v5 (ปู 1x2 ลงใน NxM)

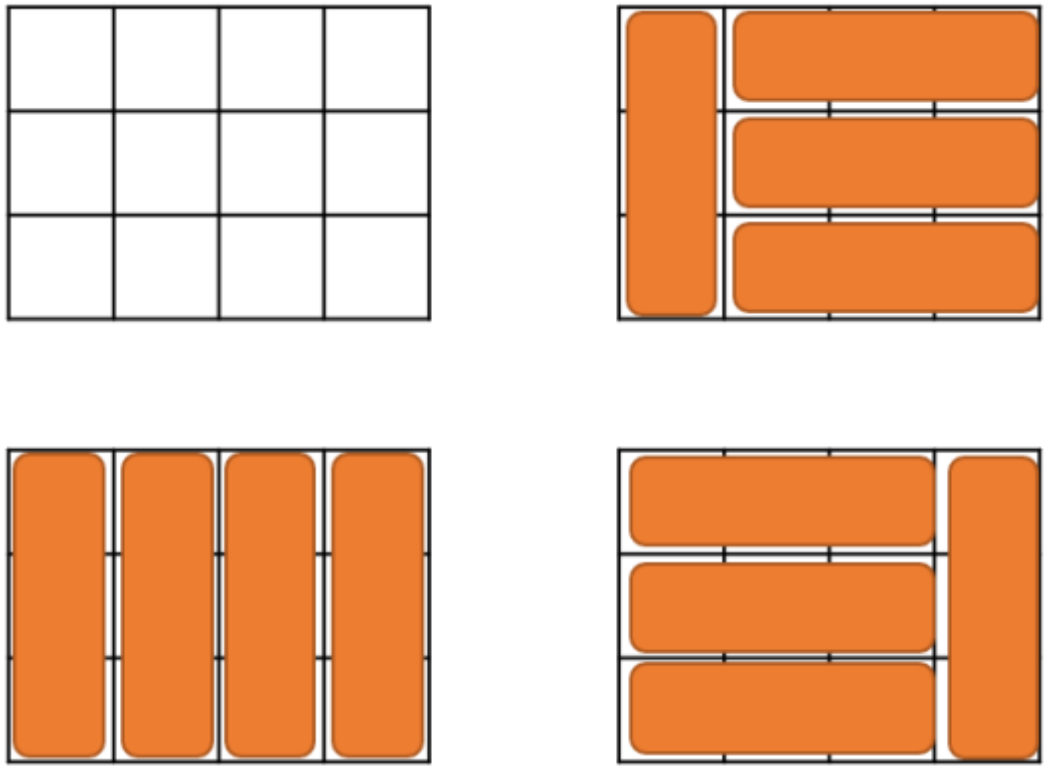
Tiling V1

กระเบื้อง V1

- **ข้อนี้มีพื้นที่ขนาด M x N ช่อง แล้วมีกระเบื้องขนาด 1xM ช่อง**
- จงหาจำนวนวิธีในการปูตารางโดยใช้กระเบื้อง โดยที่กระเบื้องสามารถถูกวางได้ในแนวตั้งหรือแนวนอนเท่านั้น เมื่อกำหนดความกว้างและยาวของตารางมาให้



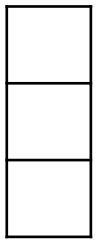
- ตัวอย่างเช่น พื้นที่ขนาด 4x3 และกระเบื้องมีขนาด 1x3 สามารถวางกระเบื้องได้ 3 แบบดังรูป



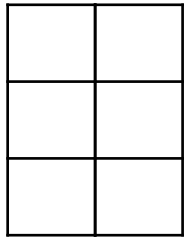
- **จากที่สังเกต เป็นโจทย์เรื่องการนับ**
  - เข้าข่ายเป็น Dynamic programming

- **แล้วเราจะเริ่มอย่างไรดี**
  - เริ่มต้นด้วยกรณีง่ายๆ ก่อน

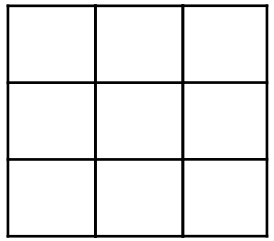
- สมมติตารางมีขนาดเป็น 3xN ดังนั้น กระเบื้องก็จะมีขนาด 3x1
- ถ้าตารางมีขนาด 3 x 1 วางได้กี่แบบ



- ถ้าตารางมีขนาด 3 x 2 วางได้กี่แบบ

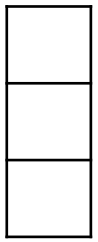


- ถ้าตารางมีขนาด 3 x 3 วางได้กี่แบบ

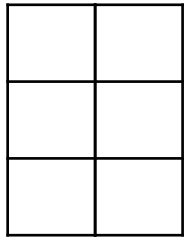


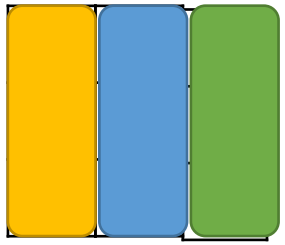
- ลองคิดดูสักครู่

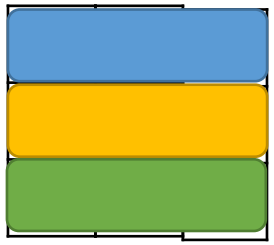
- สมมติตารางมีขนาดเป็น 3xN ดังนั้น กระเบื้องก็จะมีขนาด 3x1
- ถ้าตารางมีขนาด 3 x 1 วางได้กี่แบบ



- ถ้าตารางมีขนาด 3 x 2 วางได้กี่แบบ







- ถ้าตารางมีขนาด 3 x 3 วางได้กี่แบบ

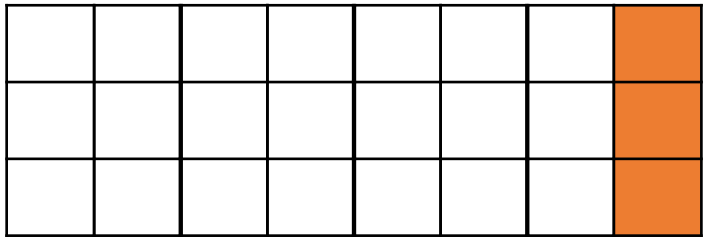
- ข้อสังเกตที่เราพบ

- ถ้าความยาวของตารางน้อยกว่าความกว้าง จะได้เพียงแบบเดียวเท่านั้น

- แต่ถ้ายาวกว่าความกว้างก็จะวางได้สองแบบคือแนวตั้งกับแนวนอน

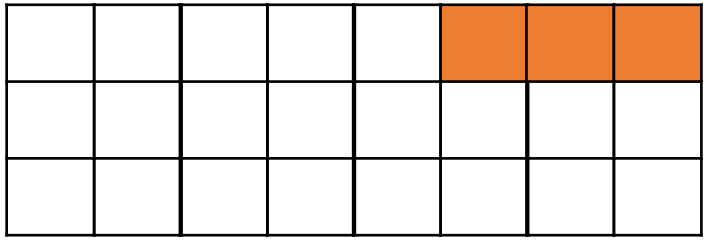
พิจารณาการวางตัวสุดท้าย วางได้อย่างไรบ้าง

- **ถ้าตัวสุดท้ายวางแนวตั้ง**



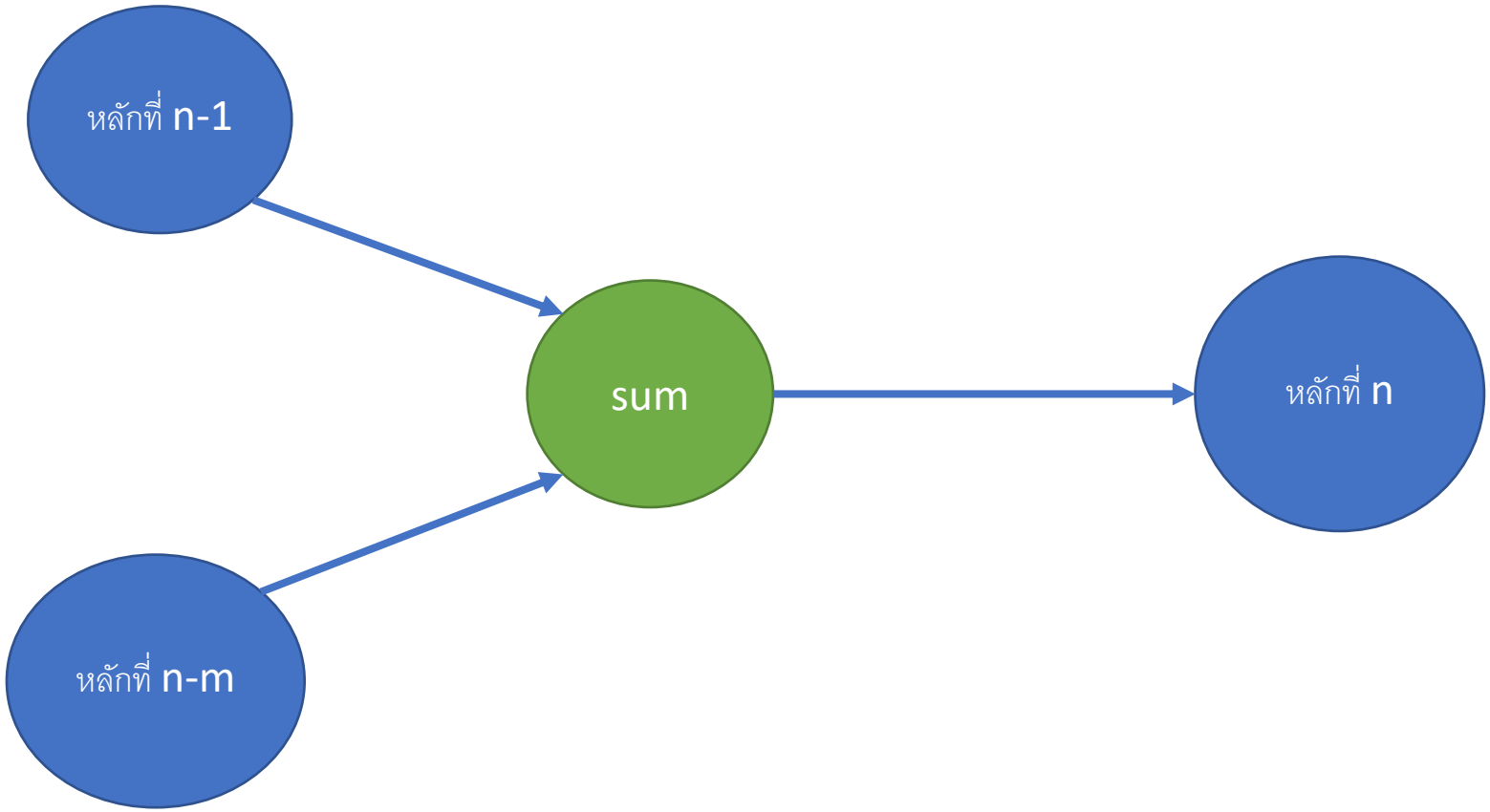
  - ไปถาม จำนวนการปูจากใครต่อ

- **ถ้าตัวสุดท้ายวางแนวนอน**



  - ไปถาม จำนวนการปูจากใครต่อ

- ดังนั้นพิจารณาหลักสุดท้าย วางได้อย่างไรบ้าง



- **เราได้กรณีทั่วไปแล้วนั่นคือ**
  - $T(n) = T(n-1) + T(n-m)$

- **Base case หละ**
  - ทีนี้ จะทำได้แค่แบบเดียวในกรณีใด

  - แล้วกรณีที่จำนวนหลัก เท่ากับ ความยาวของกระเบื้อง

```
long long counttile(int n, int m){
    if(n==m){
        return 2;
    }else if(n < m){
        return 1;
    }else{
        return counttile(n-1, m)+counttile(n-m, m);
    }
}
```

```
long long counttile(int n,int m){
    long long c[n+1];
    c[0]=0;
    for(int i=1;i<=n;i++){
        if(i>m)
            c[i] = c[i-1]+c[i-m];
        else if(i<m)
            c[i]=1;
        else
            c[i]=2;
    }
    return c[n];
}
```

Outline

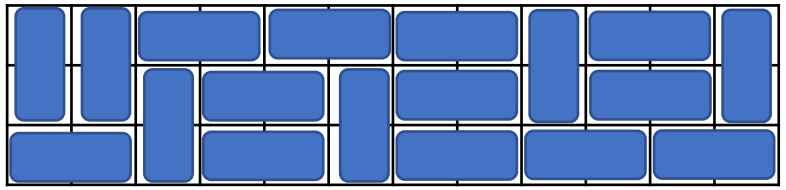
- Tiling v0 (ปู 1x2 ลงใน 2xm) ez
- Tiling v1 (ปู 1xm ลงใน nxm) ez
- Tiling v2 (ปู 1x2 ลงใน 3xn)
- Tiling v3 (tromino domino ลงใน 2xn)
- Tiling v4 (ปู เป็นสี่เหลี่ยมยาวเท่าไรก็ได้ ลงใน 2xn)
- Tiling v5 (ปู 1x2 ลงใน NxM)

Tiling V2

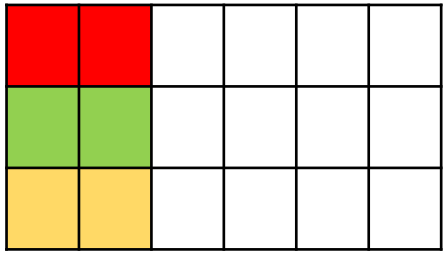
ให้ไปทำข้อ Tiling V2 ก่อน

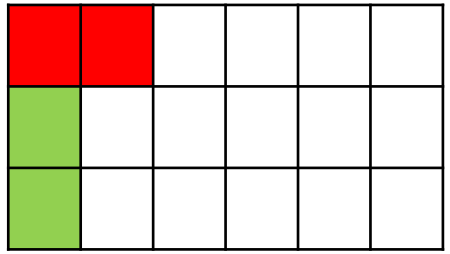
กระเบื้อง V2

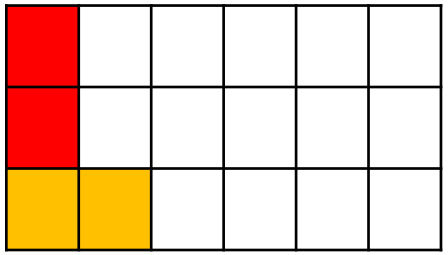
- ข้อนี้พื้นที่ขนาด 3 x N ช่อง แล้วมีกระเบื้องขนาด 1x2 ช่อง ต้องการปูถามว่าปูได้กี่วิธี



สมมติว่าด้านซ้ายคือหลักที่ n ลองปูหลักสุดท้ายดูมีแบบไหนบ้าง



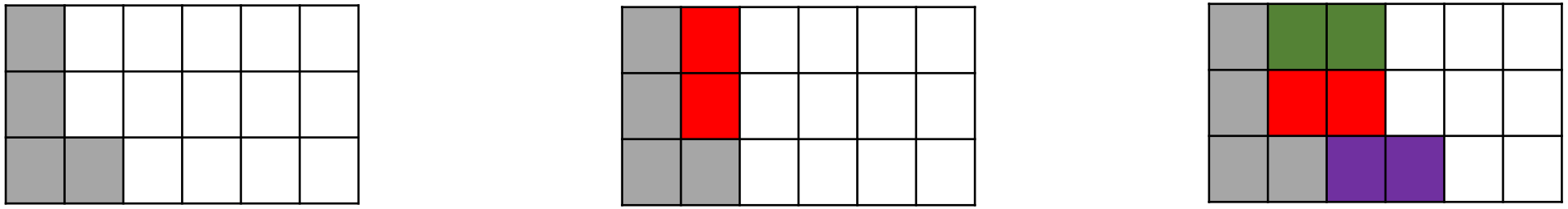




- พบว่า เอ้ย มันมีปูได้เต็ม กับปูแล้วเกินไปช่องนึง
- ปูได้เต็ม ก็ไปถาม วิธีการปูได้ของชั้น n-2 ได้นี่นา

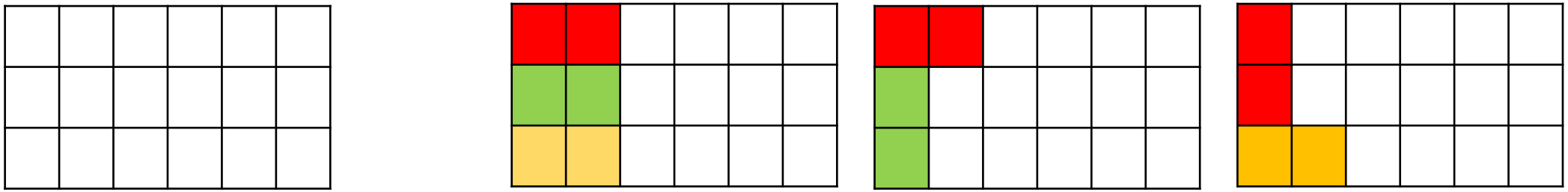
มาดูตรงที่มันเป็นติ่งมาช่องนึง

- สมมติว่า เป็นติ่งด้านล่าง แล้ววางแบบไหนได้บ้าง

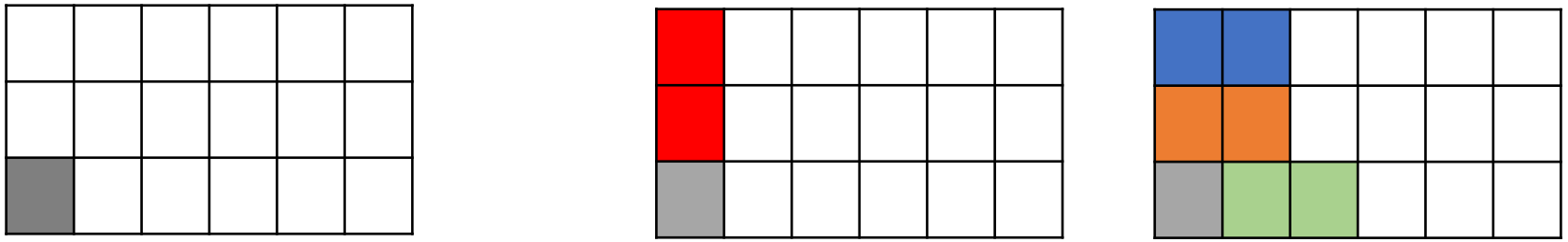


- นั่นคือถ้ามีติ่ง วางได้สองแบบ คือวางเต็มก็ไปถามช่อง n-1 ของแบบเต็ม หรือไม่งั้นก็วางแนวตั้งทำให้เป็นติ่งอีกก็ไปถามรูปแบบติ่งในช่อง n-2

- ให้ f(n) แทนจำนวนของการปูบนตารางขนาด 3xN ช่อง
- ให้ g(n) แทนจำนวนของการปูบนตารางขนาด 3XN ช่องที่มีขอบด้านหนึ่งโดนปูไปแล้ว 1 ช่อง



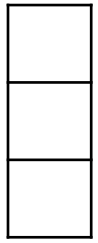
f(n) = f(n-2) + g(n-1) + g(n-1)



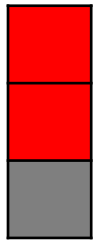
g(n) = f(n-1) + g(n-2)

Base case

- ทำไปเรื่อยๆ จนถึงไหน
- $f(0) = 1, f(1) = 0$



- $g(0) = 0, g(1) = 1$

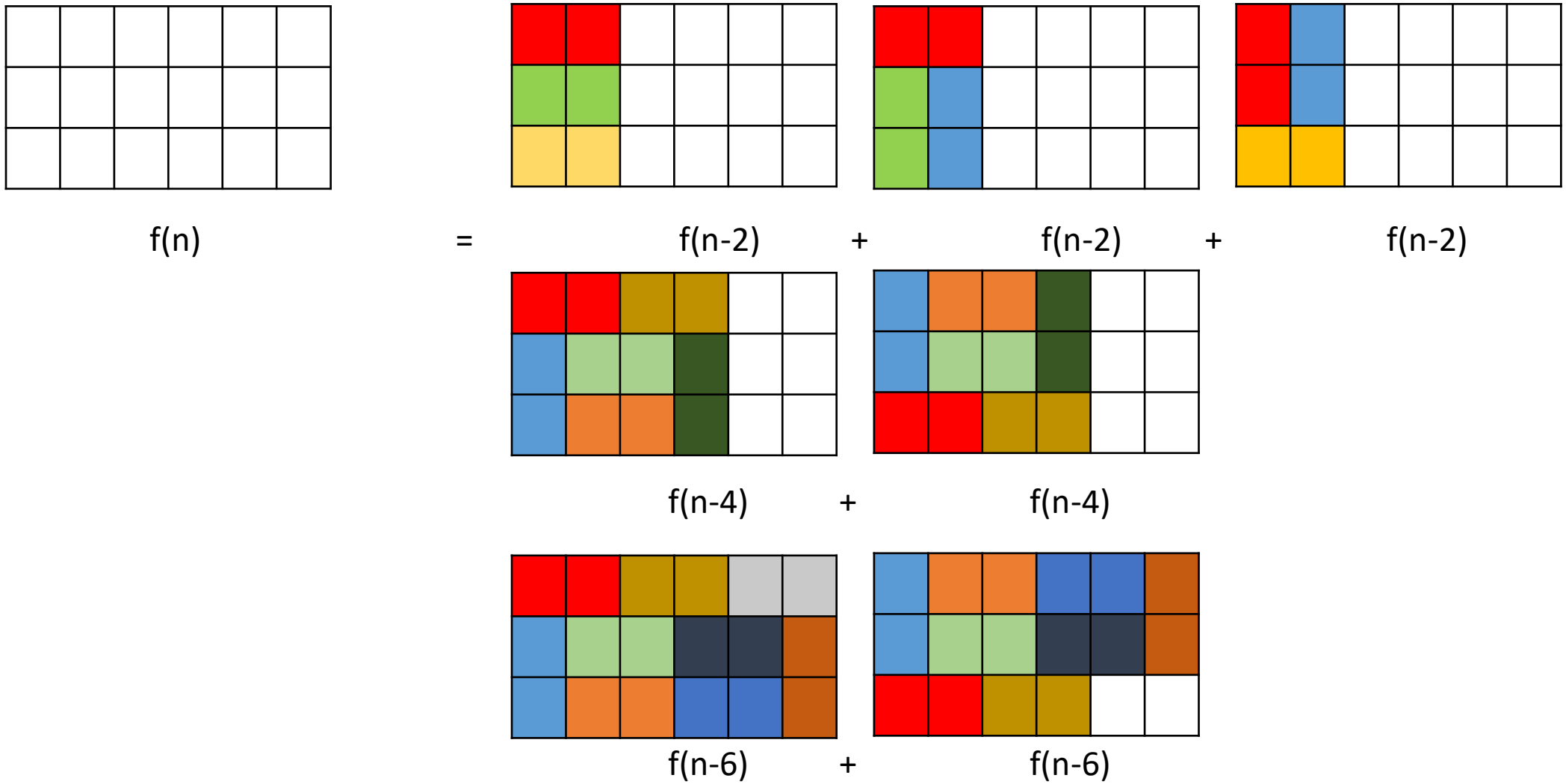


- เราก็ for loop ได้ละ

```
int f[32], g[32];
    f[0] = g[1] = 1;
    f[1] = g[0] = 0;
    for (int n = 2; n <= 30; ++n) {
        f[n] = 2 * g[n - 1] + f[n - 2];
        g[n] = f[n - 1] + g[n - 2];
    }
```

ซึ่งจริงๆ คิดเป็น **dp[][]** ก็ได้แหละ

ทั้งนี้บางคนดูความสัมพันธ์เป็นอีกแบบ



Extra note

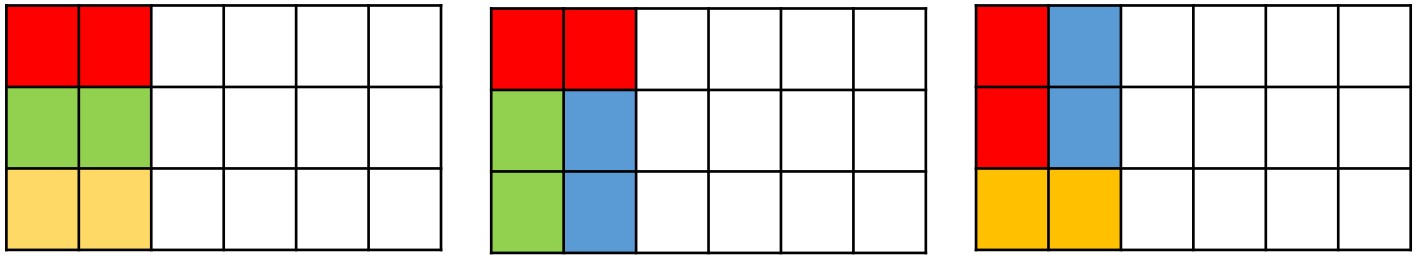
- summation 1+2+3+4+...n
- S = 1+2+3+4+...n

- เป็น pattern f(n) = 3 * f (n-2) + 2 * f (n-4) + … + 2 * f (0)
- ทีนี้จะลดรูปอย่างไรดี
- ข้างหลังมันซ้ำ 2 กันหมด พิจารณา f(n-2)

```
f(n-2) = 3*f(n-4) + 2*f(n-6) + ... + 2*f(0)
f(n)-f(n-2) = 3*f(n-2) + 2*f(n-4) + ... + 2*f(0) -(3*f(n-4) + 2*f(n-6) + ... + 2*f(0))
f(n)-f(n-2) = 3*f(n-2) -f(n-4)
```

f(n) = 4*f(n-2)-f(n-4) นั่นเอง

- ทีนี้ก็พิจารณา base case
- f(0) = 1
- f(1) = 0
- f(2) = 3



```
void init() {
    dp[0] = 1;
    dp[2] = 3;
    for(int i=4; i<=30; i+=2) {
        dp[i] = 4*dp[i-2] - dp[i-4];
    }
}
```

Outline

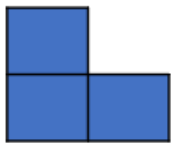
- Tiling v0 (ปู 1x2 ลงใน 2xm) ez
- Tiling v1 (ปู 1xm ลงใน nxm) ez
- Tiling v2 (ปู 1x2 ลงใน 3xn)
- Tiling v3 (tromino domino ลงใน 2xn)
- Tiling v4 (ปู เป็นสี่เหลี่ยมยาวเท่าไรก็ได้ ลงใน 2xn)
- Tiling v5 (ปู 1x2 ลงใน NxM)

Tiling V3

ให้ไปทำข้อ Tiling V3 ก่อน

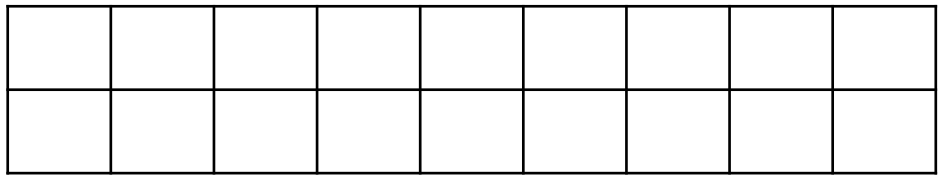
ปูกระเบื้อง v3

- คราวนี้มีกระเบื้อง 2 แบบ



Tromino

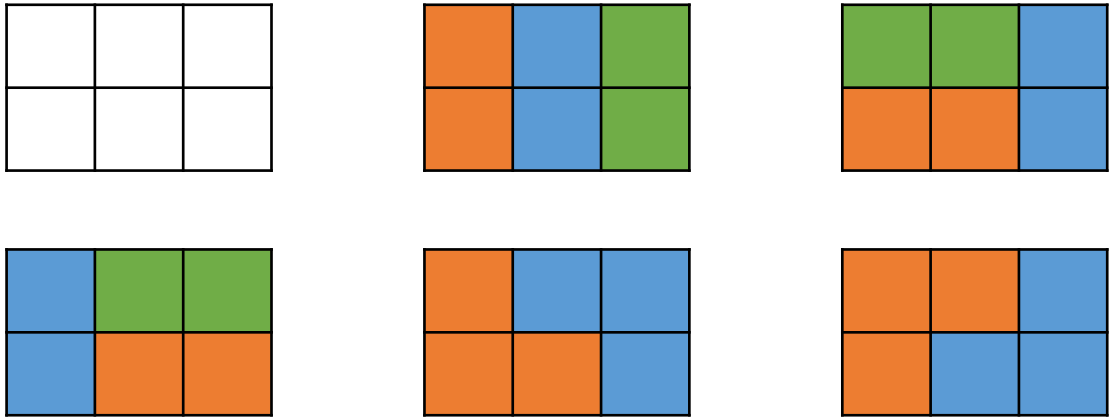
Domino



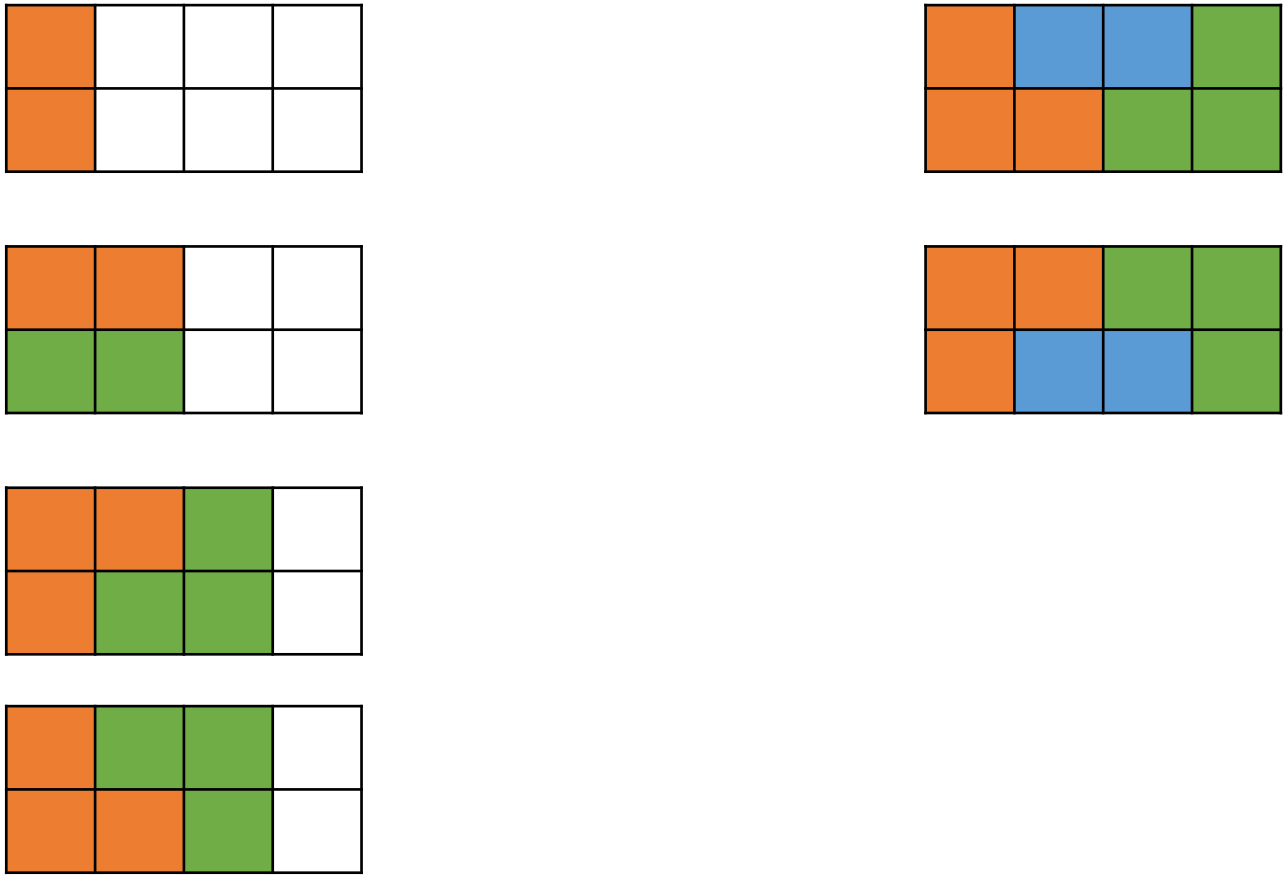
- ส่วนพื้นที่ต้องปูมีขนาด 2x N
- คำตอบให้ mod 10^9+7

กรณี n = 1 และ n = 2

กรณี n = 3



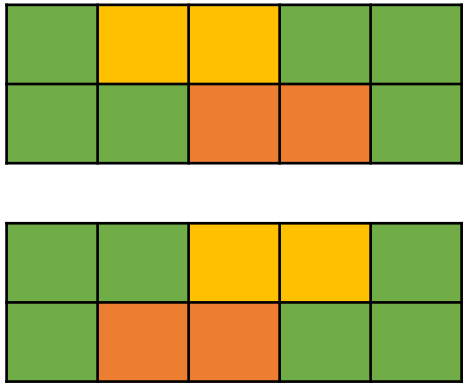
กรณี n = 4



นั่นคือ dp[4] = dp[3] + dp[2] + 2 dp[1] + 2 dp[0]

กรณี n=5

- คล้าย n = 4 แต่มี 2 กรณีนี้เพิ่มมา



- นั่นคือ pattern ด้านหลังมี 2 แบบ

- จากที่เห็นว่า pattern ตั้งแต่ 3 เป็นต้นไปมี 2 แบบ

dp[n] = dp[n-1]+dp[n-2]+2(dp[n-3]+dp[n-4]+...dp[0])----------(1)

คล้ายๆ เดิม ถ้าดู dp[n-1]

dp[n-1] = dp[n-2]+dp[n-3]+2(dp[n-4]+dp[n-5]+...dp[0])-------(2)

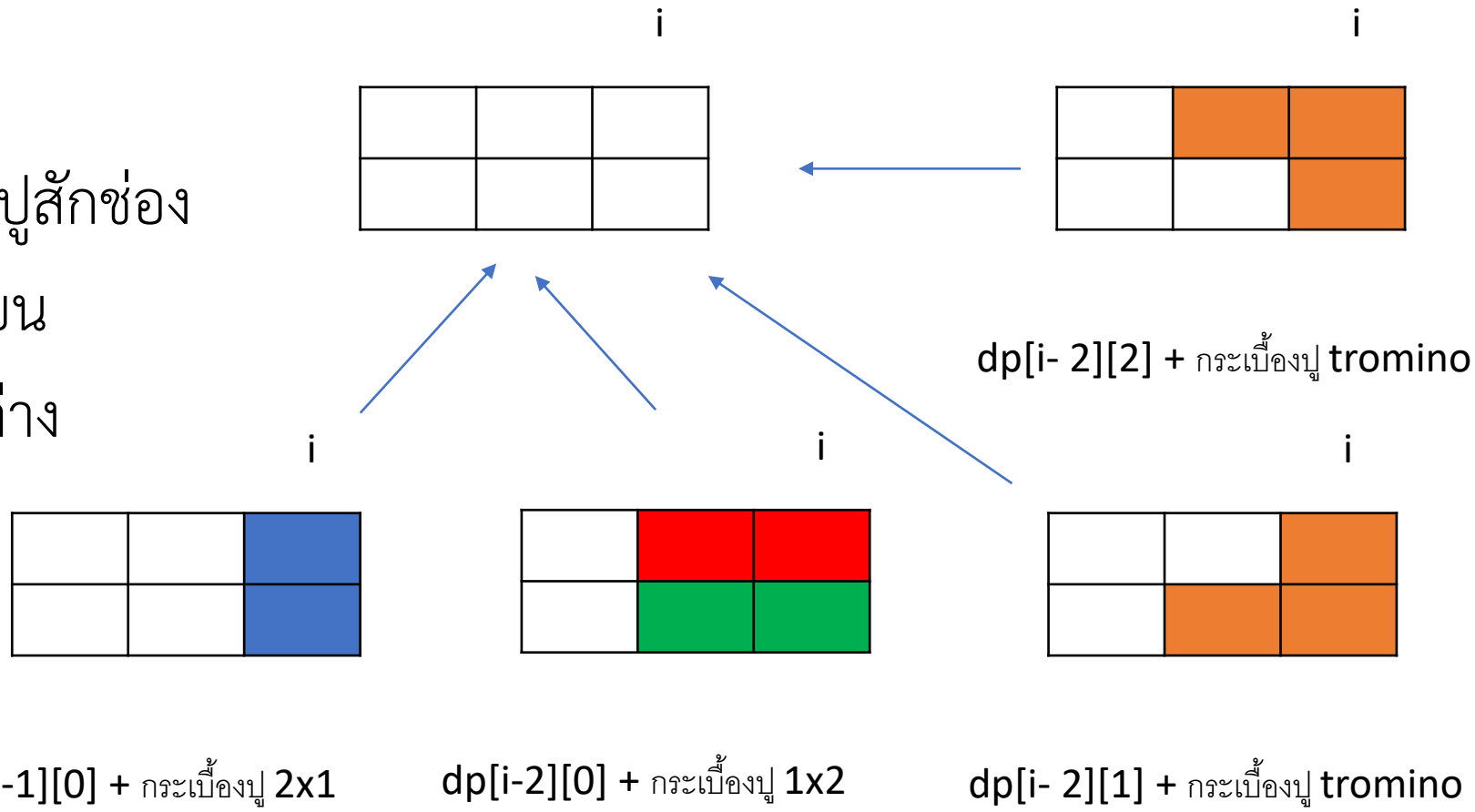
เอา (1)-(2)

dp[n]- dp[n-1] = dp[n-1]+dp[n-3]

dp[n] = 2dp[n-1]+dp[n-3]

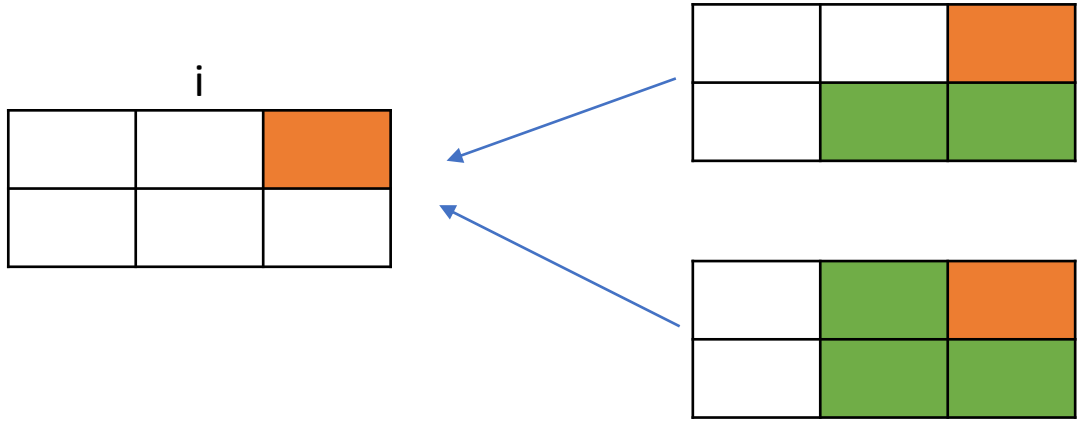
ถ้าทำเป็น DP 2 มิติ

- เราจะเก็บ state ว่าตอนนี้อยู่colที่เท่าไร แล้วก็มีส่วนที่เลยมาหน้าตาเป็นอย่างไร
- dp[i][j]
  - เมื่อ j = 0 คือไม่มีปูสักช่อง
  - j = 1 คือว่างช่องบน
  - j = 2 คือว่างช่องล่าง



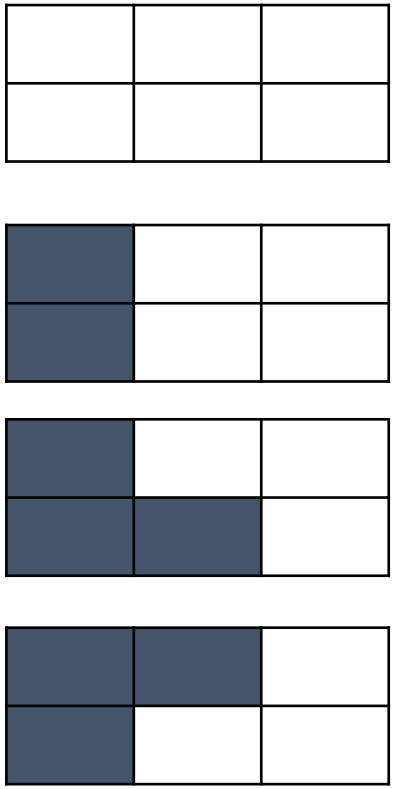
- ทีนี้ dp[i][x] แต่ละอันสร้างมาจากอะไรได้บ้าง

- dp[i][0] = dp[i-1][0]+dp[i-2][0]+dp[i-2][1]+dp[i-2][2]
- dp[i][1] = dp[i-1][0]+dp[i-1][2]
- dp[i][2] = dp[i-1][0]+dp[i-1][1]



Basecase

- dp[0][0] = 1
- dp[1][0] = 1
- dp[1][1] = 1
- dp[1][2] = 1

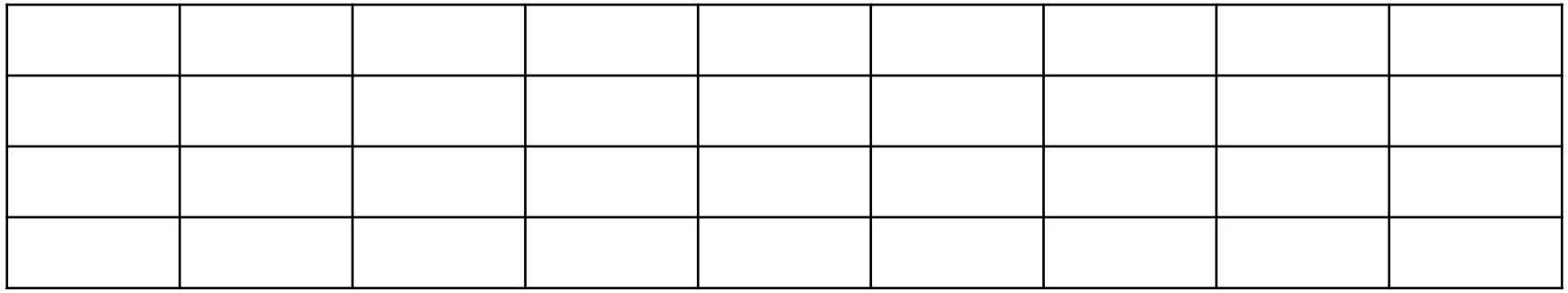


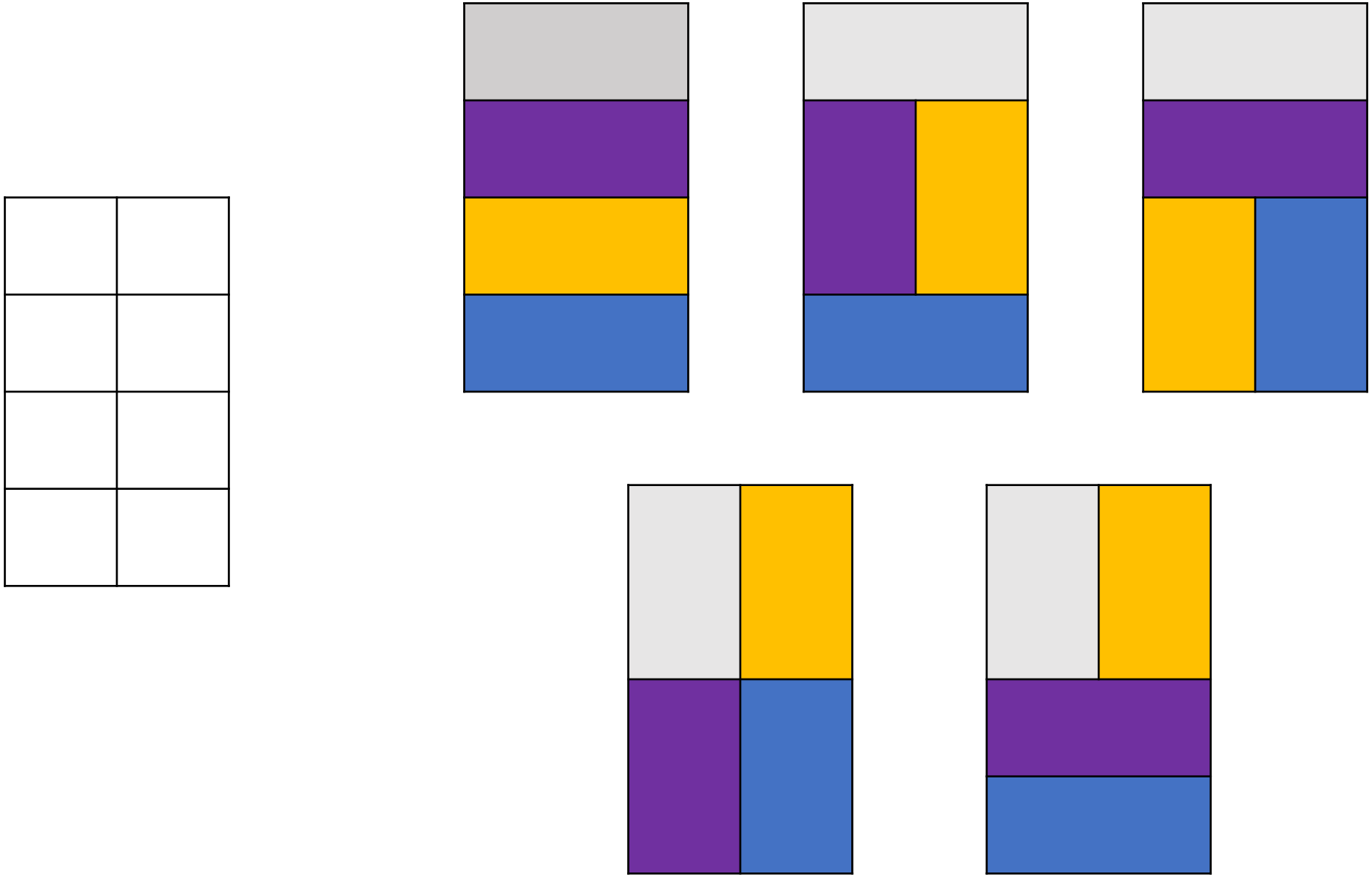
- จากนั้นก็ for ตั้งแต่ 2 ถึง n แล้วก็ตอบช่อง dp[n][0]
  ออ อย่าลืม mod

Tiling V3.5

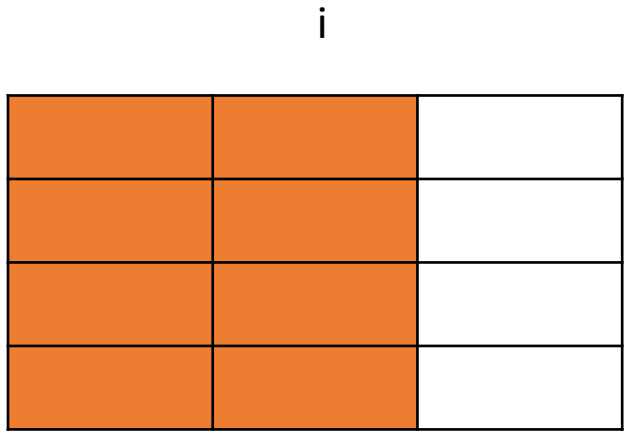
ลองคิดดูถ้า ตารางขนาด 4xn แล้วปูด้วยกระเบื้องขนาด 1x2

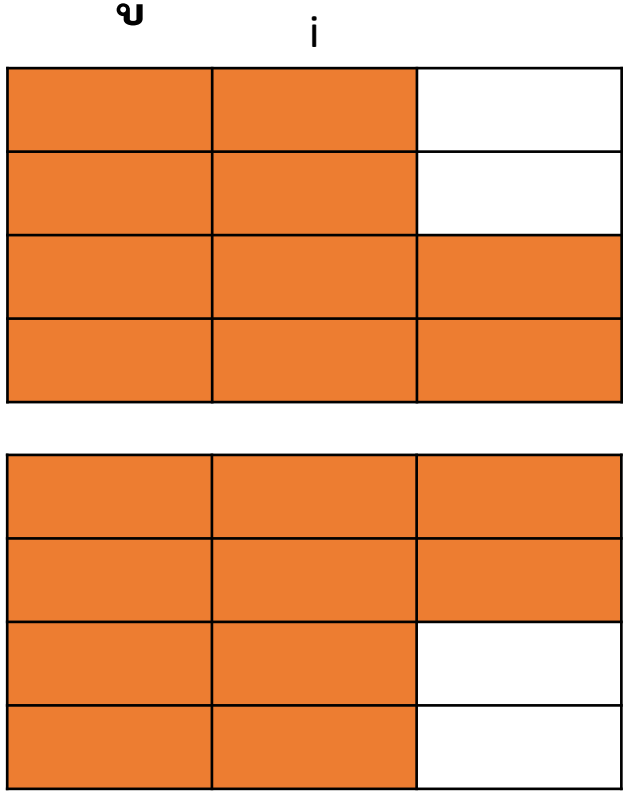
- ข้อนี้พื้นที่ขนาด 4 x N ช่อง แล้วมีกระเบื้องขนาด 1x2 ช่อง ต้องการปูถามว่าปูได้กี่วิธี
- $1 \leq N \leq 1000$

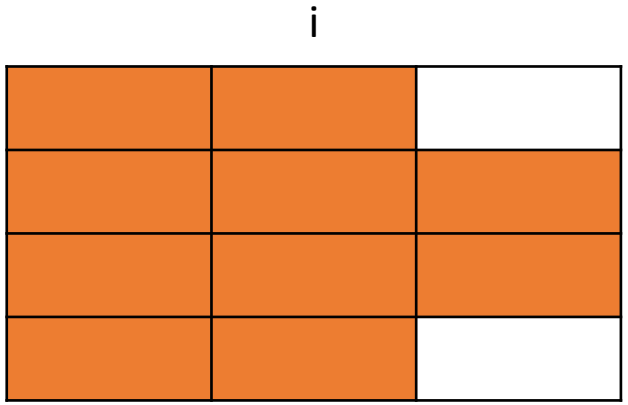




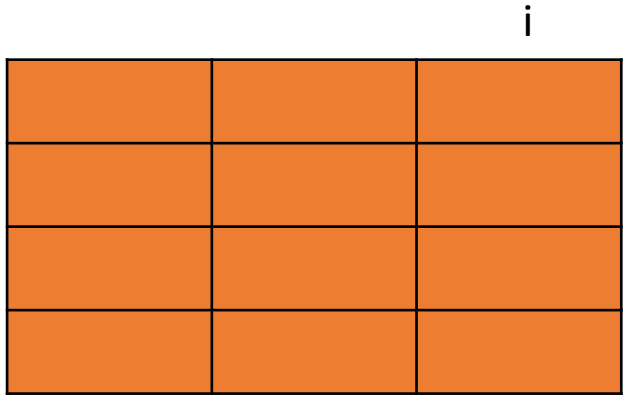
- ถ้าให้ F(i) แทนปูจนเต็ม col i พอดี
- G(i) แทน เต็ม col i แล้วมีปูเกินมาช่องว่าง 2 อันติดกันบนล่าง
- H(i) แทน เต็ม col i แล้วมีปูเกินมาช่องว่าง 2 อันติดกันตรงกลาง

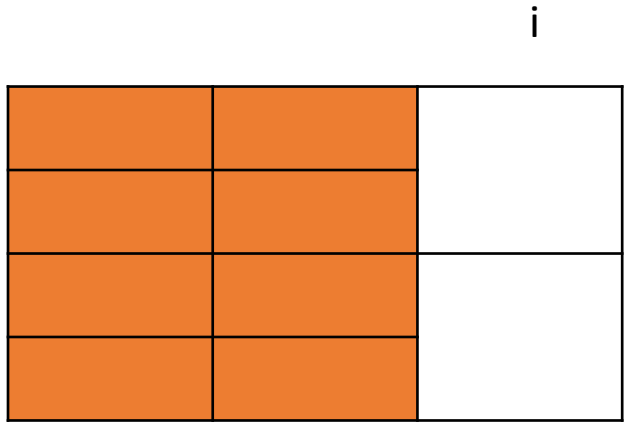


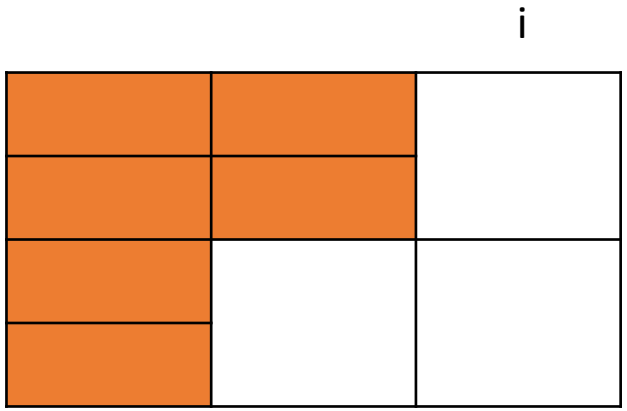


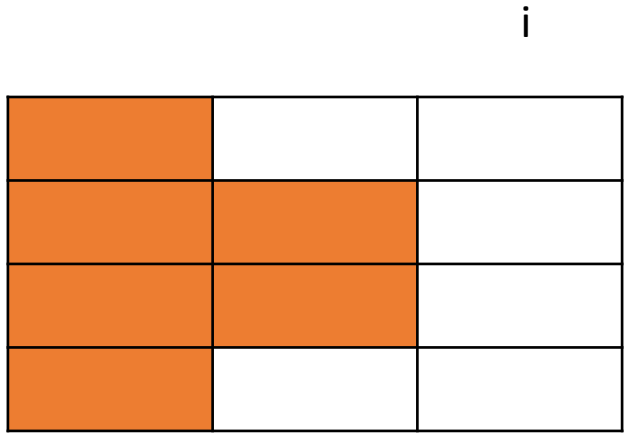


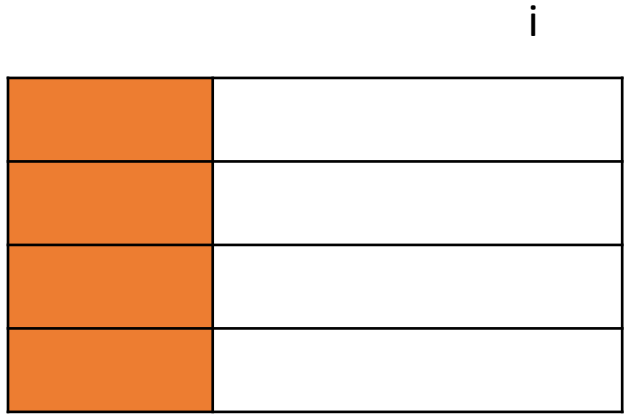
- $F(n) = F(n-1) + F(n-2) + G(n-2) + G(n-2) + H(n-2)$

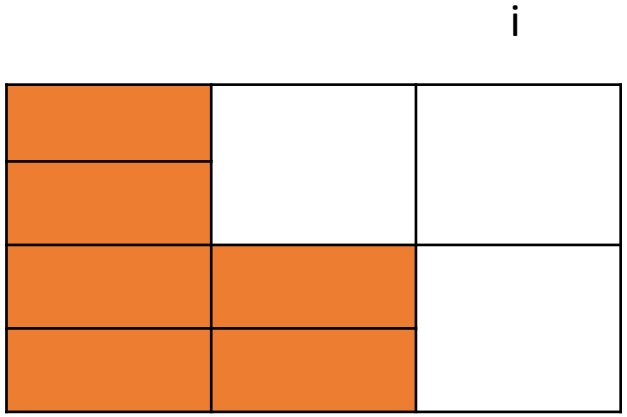




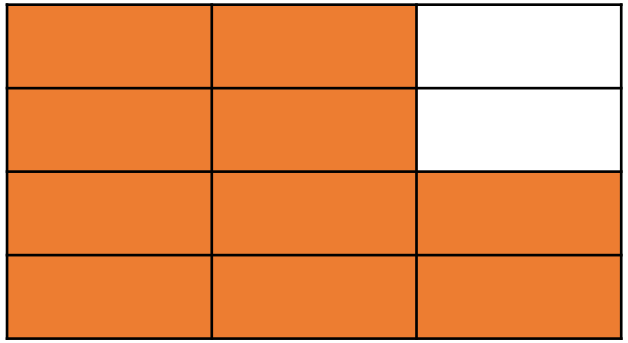


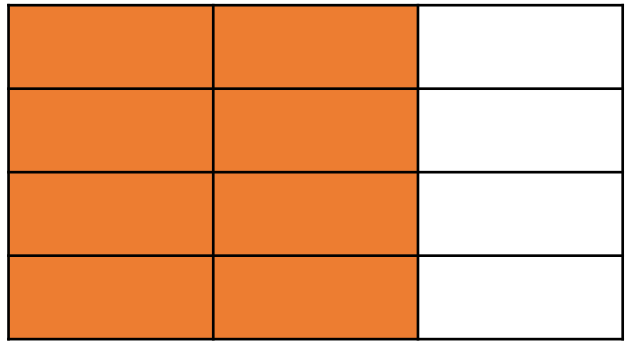


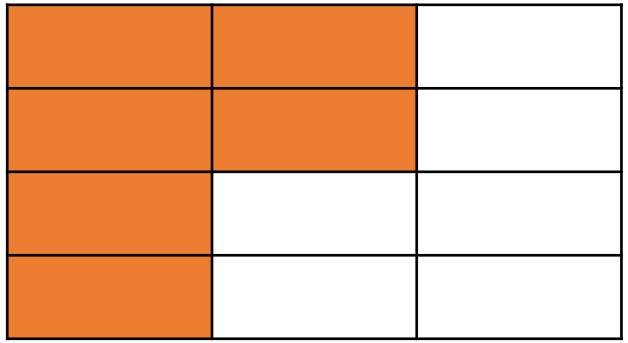




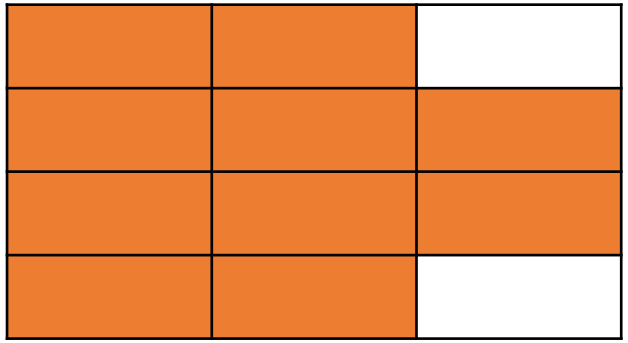
- $g(n) = f(n)+g(n-1)$

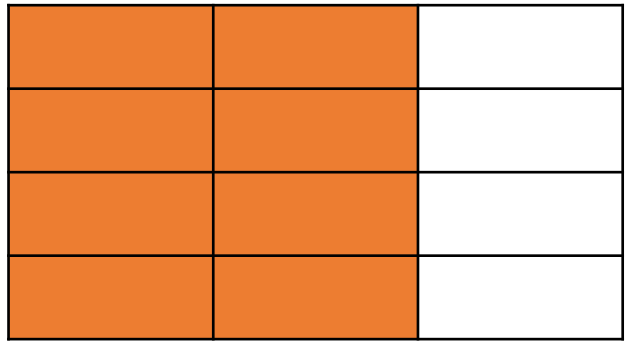


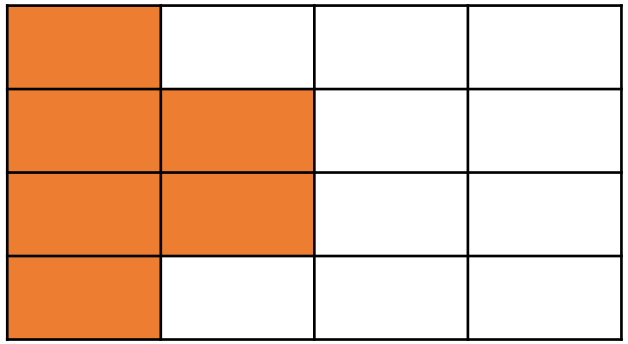




- $h(n) = f(n)+h(n-2)$







base case

```
f(0) = 1
f(1) = 1
g(0) = f(0) + g(-1) = 1 + 0 = 1
g(1) = f(1) + g(0) = 1 + 1 = 2
h(0) = f(0) + h(-2) = 1 + 0 = 1
h(1) = f(1) + h(-1) = 1 + 0 = 1
```

```
for(int i=2;i<=1000;i++){
        F[i] = F[i-2]+F[i-1]+G[i-2]*2+H[i-2];
        G[i] = F[i]+G[i-1];
        H[i] = F[i]+H[i-2];
    }
```

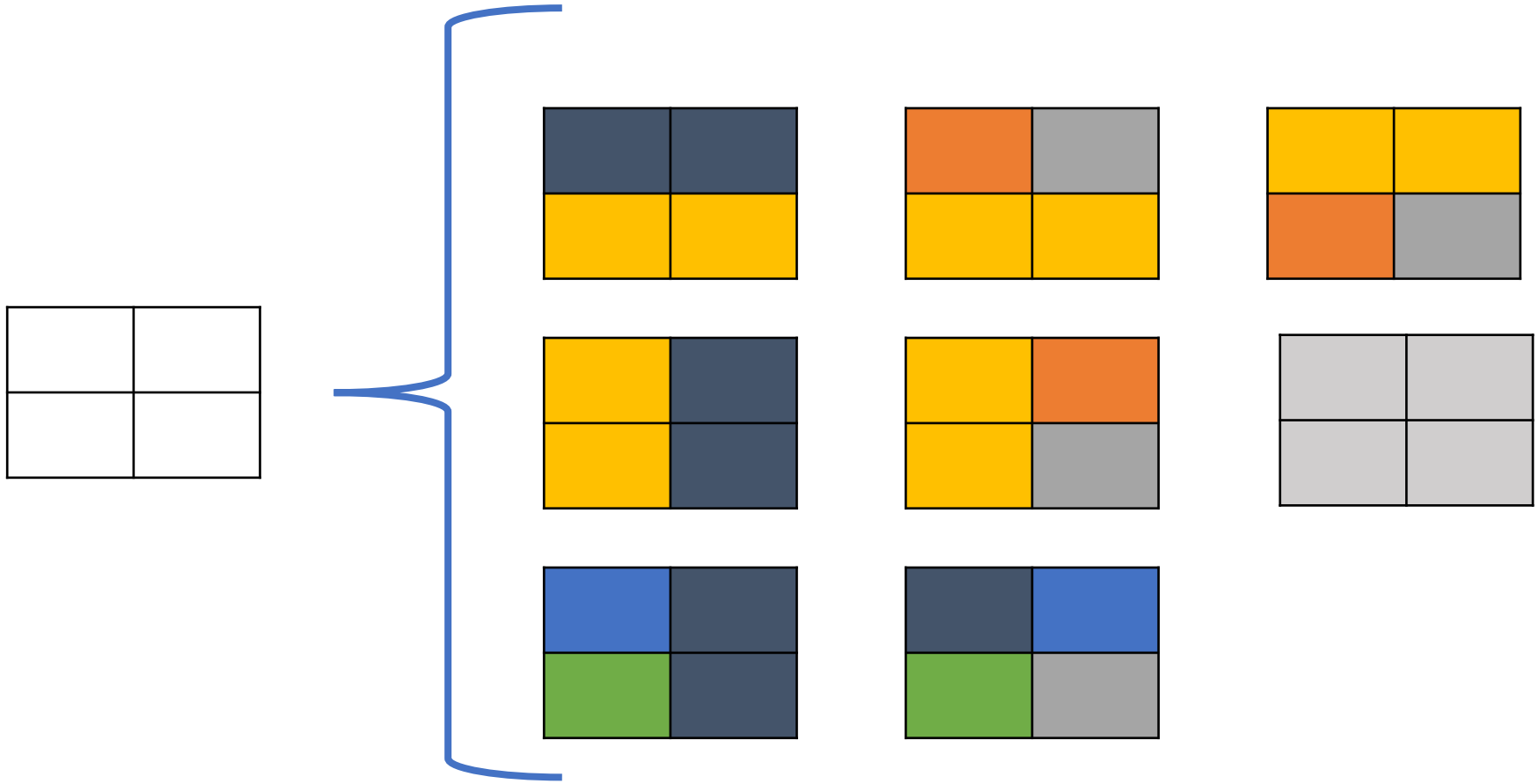
- ได้ closed form เป็น

- $f(n)=f(n-1)+5f(n-2)+f(n-3)-f(n-4)$
- ลองไปคิดดู

Tiling V4

ให้ไปทำข้อ Tiling V4 ก่อน

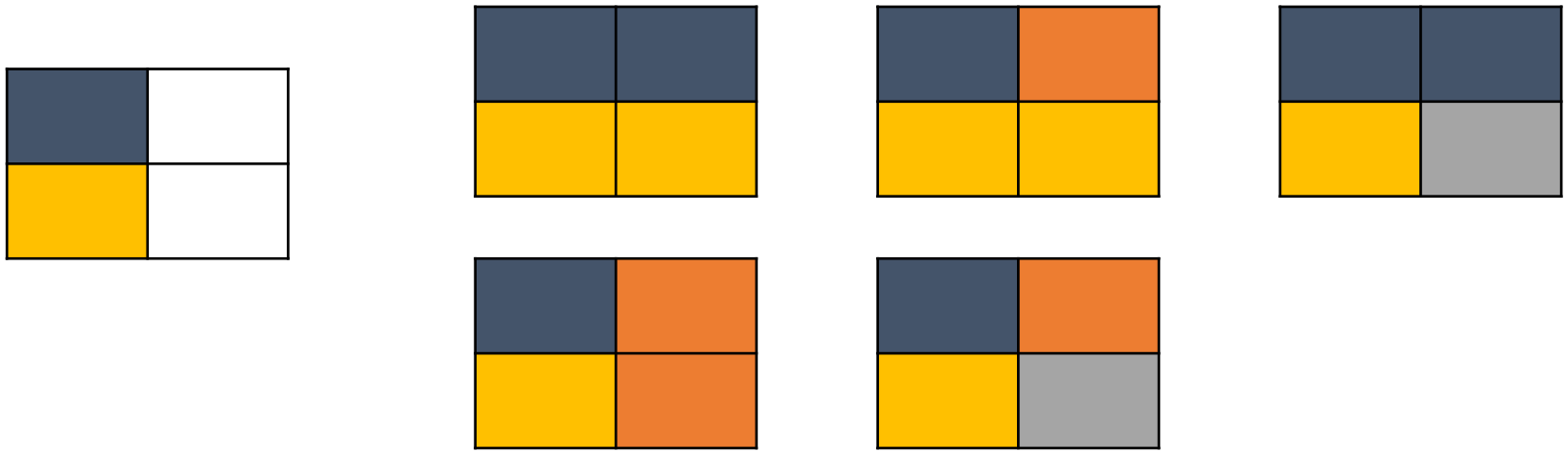
Tiling ตาราง 2Xn ที่ใช้กระเบื้องต่อเป็นสี่เหลี่ยมยาวเท่าไรก็ได้



- **Key คือ N=2 นี้เลย**
- **ก็จะมีหลักๆ 2 กรณีคือ**
  - ตอนนี้ชั้นสุดท้ายเป็นช่องแยกกัน
  - ชั้นสุดท้ายรวมกัน

ชั้นสุดท้ายเป็นช่องแยกกัน

- สมมติให้ dp[i][1] = ชั้นสุดท้ายแยกกัน
- ก่อนหน้าเป็นอะไรได้บ้าง



- dp[i][1] = 4*dp[i-1][1]+dp[i-1][2]

ชั้นสุดท้ายรวมกัน

- สมมติให้ dp[i][2] = ชั้นสุดท้ายรวมกัน
- ก่อนหน้าเป็นอะไรได้บ้าง

- dp[i][2] = d[i-1][1] + 2*dp[i-1][2]

base case

- ก็ต้องทำให้มันมีค่าตอนท้ายในกรณีเล็กสุด
- dp[1][1] = dp[1][2] = 1;
- นั่นเอง
- md = 1e9+7

```
for (int i = 2; i <= 1000; i++) {
        dp[i][1] = (dp[i-1][1]*4%md + dp[i-1][2])%md;
        dp[i][2] = (dp[i-1][1] + dp[i-1][2]*2%md)%md;
    }
```

- ทีนี้คำตอบคืออะไร
- ก็ต้องเป็น 2 กรณีรวมกันนั่นเอง

- cout<<(dp[n][1] + dp[n][2])%md<<endl;

Tiling V5

ให้ไปทำข้อ Tiling V5 ก่อน

Tiling ตารางแบบ NxM ด้วย กระเบื้อง 2x1

- ปัญหา จงหาวิธีในการปูกระเบื้องลงบนตารางขนาด NxM ด้วยกระเบื้องขนาด 2x1 ให้เต็มพื้นที่และไม่มีกระเบื้องซ้อนทับกัน
- Constraint
  - $1 \leq N \leq 9$
  - $1 \leq M \leq 1000$

Solution

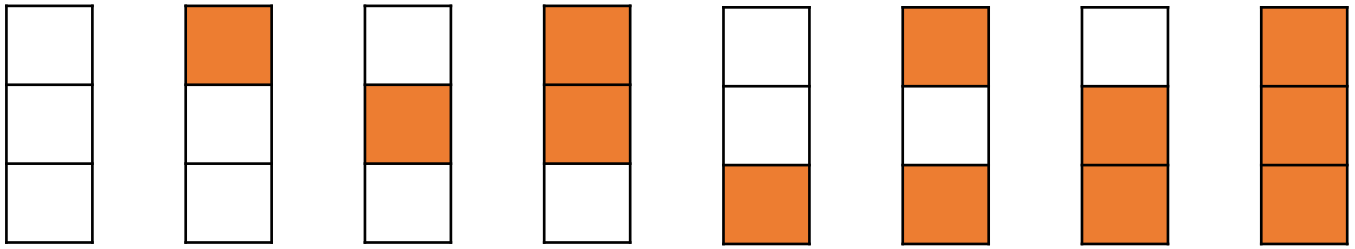
- เคสง่าย ก็คือที่เราทำกันมาก่อนหน้านี้ เราก็พยายามหา recurrence relation สำหรับค่า N ที่ได้รับมา ตัวอย่างเช่น
- N = 2
  - ก็ได้ F(M) = F(M-1) + F(M-2)
- N = 3
  - ก็ได้ F(M) = 4F(M-1)-F(M-2)
- แต่ว่าเมื่อค่า N มากขึ้น หา recurrence relation ยาก!!

เราก็ใช้วิธีอื่น

- นั่นคือ เราจะนับรูปแบบตามหน้าตาของ column แทน

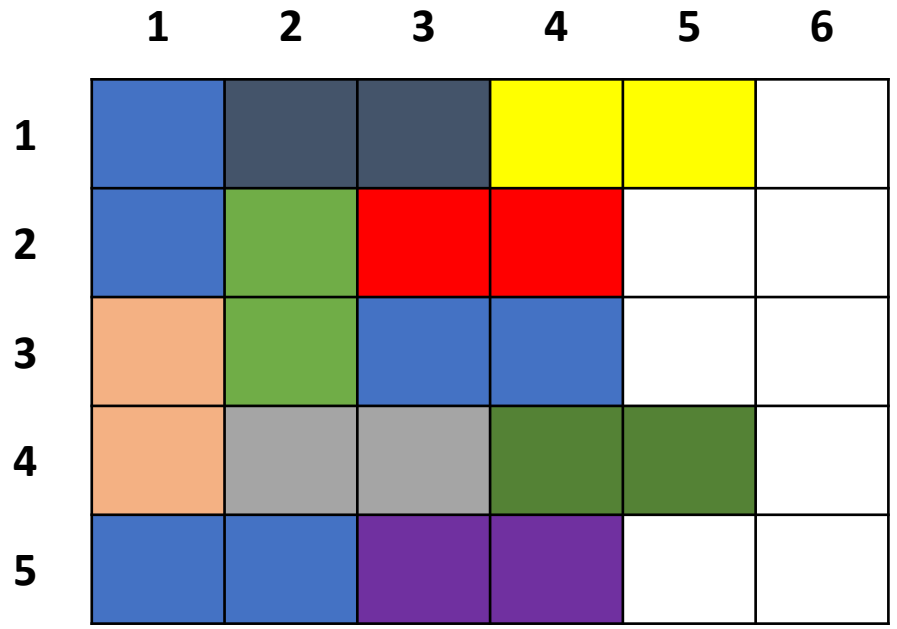
- แล้ว column หน้าตาเป็นอย่างไรได้บ้าง ก็แต่ละช่องมีกระเบื้องหรือไม่มี

- ดังนั้น Bit mask ต้องมาแล้ว
  - แทนหน้าตาของกระเบื้องในแถวนั่นเอง

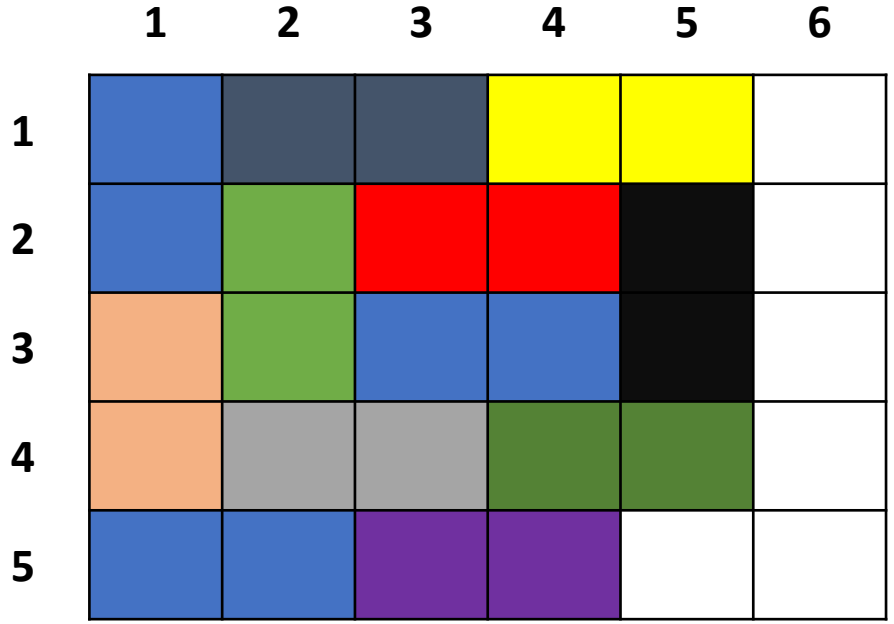


- **เรานิยามหน้าตาของกระเบื้องใน column เป็น n-bit integer โดยที่**
  - ถ้า bit ที่ i เป็น 1 นั่นคือมีกระเบื้องปูในช่องที่ i (occupied)
  - ถ้า bit ที่ i เป็น 0 นั่นคือไม่มีกระเบื้องปูในช่องที่ i

- **นิยาม dp[i][p] (based 1) เป็นจำนวนวิธีในการปูจนเต็ม i column และมีหน้าตา p ใน column i+1**



แบบนี้นับให้สำหรับกรณีที่ปูเต็ม 4 column แล้วก็มีหน้าตา เป็น 9(01001)



แบบนี้ไม่นับให้สำหรับกรณีที่ปูเต็ม 4 column แล้วก็มีหน้าตา เป็น 15 เพราะว่า สีดำไม่ได้ถูกใช้ใน 4 column แรก

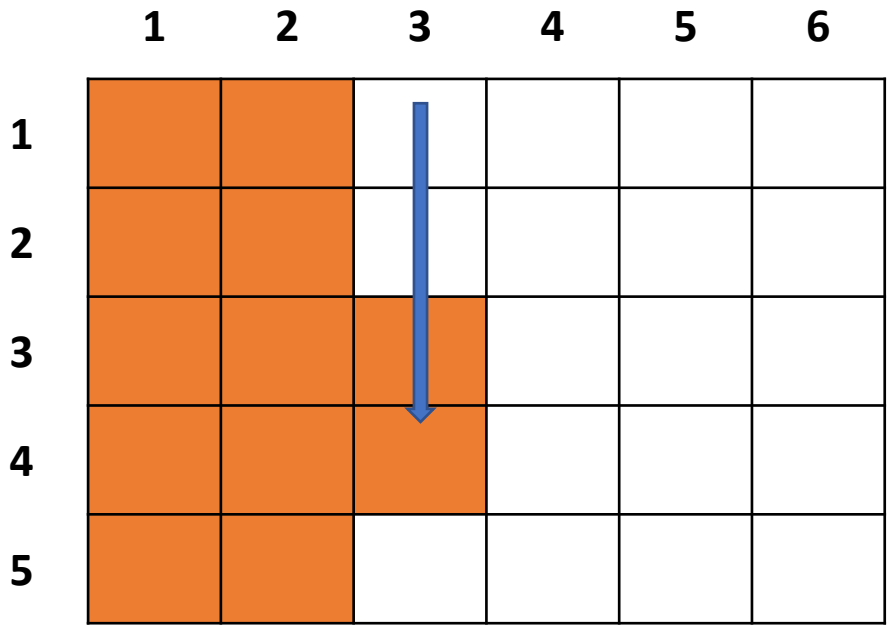
- แล้วเราจะหาความสัมพันธ์ระหว่าง state ได้อย่างไร

- วิธีหนึ่งในการที่จะได้ค่า dp[i][p] จาก dp[i-1][q] โดยการไปหา sum ของทุกๆ dp[i-1][q] ที่พบว่ามีการปูกระเบื้องใน column ที่ i แล้วทำให้ col i+1 มีหน้าตา p

- Time ในการคำนวณ dp[i][p] จะเท่ากับ $O(2^N)$ และจะมี state $mx2^N$ ดังนั้น Time complexity เป็น $O(m2^N 2^N)$ ช้ามาก

- เพื่อหลีกเลี่ยงการหาทุกวิธีของหน้าตา q ที่ทำให้ dp[i][q] สร้าง dp[i+1][p] ได้เมื่อกำหนด p มา

- เราจะมองกลับด้าน ก็คือ dp[i-1][q] สามารถสร้าง dp[i][p] อะไรได้บ้าง ก็ไปบวกค่า dp[i-1][q] ให้ dp[i][q]
- (นั่นคือ เราจะไปตรวจสอบทุกวิธีในการเติม col ที่ i ที่มีหน้าตา q)
- วิธีการทำก็คือ ใน column ที่ i เราก็วิเคราะห์จากช่องแรกไปช่องสุดท้ายของ column

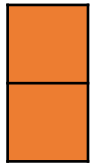


พิจารณา col ที่ i-1 ด้วยหน้าตา q

ถามว่าไปเพิ่มจำนวนนับ dp[i][p] p ตัวไหนบ้าง

เราก็ scan จากบนลงล่าง

เริ่มด้วย col ที่ i ที่มีหน้าตา q

- **ทีนี้ระหว่าง scan ถ้า ช่องที่ j มีการปูกระเบื้องแล้ว ก็ไม่ทำอะไร**
  - ไปทำช่องถัดไปต่อ
- **แต่ว่าช่องที่ j ว่างสิ่งที่เราทำได้คือ ปูกระเบื้อง ทีนี้มันปูได้กี่แบบ**
  - ปูช่องที่ (i,j) และช่องที่ (i+1,j) และมาร์คว่าปูแล้ว
  - ปูช่องที่ (i,j) และช่องที่ (i+1,j+1) และมาร์คว่าปูแล้วถ้า ช่องที่ j+1 ยังไม่ถูกปู 
- **แต่ละแบบก็จะสร้างหน้าตา p ใน col i+1 ต่อไป**
  - ดังนั้นเราก็เพิ่ม dp[i][q] ให้กับ dp[i+1][p]

Base case

- ทีนี้เราสังเกตได้ว่า สำหรับกรณี i = 0 นั่นคือเราก็ต้องให้เริ่มต้นตารางเปล่าเป็น 1
- นั่นคือหน้าตา i+1 เป็นแบบไหน
- ต้องว่างแหละ

ดังนั้นจะได้ว่า dp[0][p] จะเป็น 1 ถ้า p เป็น 0

จะเป็น 0 กรณีอื่น

- และเมื่อเราปูกระเบื้องเต็มแล้วจะเอาคำตอบไหนมาตอบ
- นั่นคือเราปูเต็ม col ที่ m แล้ว แล้วหน้าตา col m+1 แบบไหนถึงเป็นคำตอบเรานั่นเอง

- ดังนั้นคำตอบเราคือ dp[m][0]

```
int dp[1000+1][(1<<9)];
int n,m;
bool occupied(int i, int q){
    return q&(1 << (i-1));
}
void search(int i, int j, int p, int q){
    if(i == n+1){
        dp[j+1][p] += dp[j][q];
        return;
    }
    if(occupied(i, q)){
        search(i+1, j, p, q);
        return;
    }
    if(i+1 <= n && !occupied(i+1, q)){
        search(i+2, j, p, q);
    }
    if(j+1 <= m){
        search(i+1, j, p^(1 << (i-1)), q);
    }
}
```

```
int main(){
    memset(dp, 0, sizeof(dp));
    dp[0][0] = 1;
    cin>>m>>n;
    for(int j = 0; j < m; j++){
        for(int q = 0; q < (1<<n); q++){
            search(1, j, 0, q);
        }
    }
    cout<<dp[m][0]<<endl;
    return 0;
}
```